

HalluGuard : Détection et Mitigation des Hallucinations

dans les Pipelines RAG Agentiques

par un Middleware NLI Multi-Couches

Projet 15 — Module Représentation et Résolution de Problèmes en Intelligence Artificielle
ENSAM Meknès — 4^e Année Génie Informatique — 2025-2026

Souleymane Diallo & Marc Thierry Nankouli

s.diallo@edu.umi.ac.ma

Encadrant : Prof. Hajji Tarik

Mai 2026

Résumé

Les pipelines RAG (*Retrieval-Augmented Generation*) agentiques réduisent les hallucinations des grands modèles de langage (LLM) de 30 à 60 %, mais les 40 à 70 % de sorties erronées restantes se propagent librement à travers chaque nœud du pipeline sans aucun mécanisme d'interception natif. Nous présentons **HalluGuard**, un middleware de détection et de mitigation des hallucinations pour pipelines LangGraph à quatre nœuds (`retrieval` → `reasoning` → `tool_call` → `generation`). HalluGuard combine cinq composants : un vérificateur NLI inter-sources (M1, `cross-encoder/nli-MiniLM2-L6-H768`, 117 M paramètres, pré-entraîné sur MNLI), un BeliefState temporel persistant (M2, 50 faits maximum, éviction par score confiance × récence), un graphe de dépendances acyclique dirigé (DependencyDAG), un moteur de politique d'action (PolicyEngine) et un serveur MCP (Anthropic, 2024) exposant quatre outils standardisés. Nous proposons également une taxonomie originale de cinq types d'hallucinations agentiques (T1–T5), dont deux types (T4 causale, T5 délégation) sont absents de la littérature HaluEval standard. Sur un benchmark de 60 scénarios HaluEval-Agentic couvrant les cinq types, HalluGuard atteint **85,0 % de taux de détection** (contre 0 % pour le pipeline baseline), un overhead moyen de **278 ms sur CPU standard**, un PBR@1 de **85,0 %** et un delta de détection de **1,03 étape**. L'étude d'ablation montre que M1 porte 90 % du gain total (+76,7 points) et que M2 apporte un gain décisif sur les hallucinations temporelles T2 (83,3 % → 100,0 %). Les 23 tests unitaires passent en 13,58 s. Le système est déployé entièrement en local, sans GPU, sans fine-tuning et sans API payante.

Mots-clés : hallucination LLM, RAG agentique, inférence de langage naturel, LangGraph, MCP, BeliefState, middleware de vérification, HaluEval, A2A

Résumé

Abstract. Agentic RAG (Retrieval-Augmented Generation) pipelines reduce LLM hallucinations by 30–60%, yet the remaining 40–70% of erroneous outputs propagate unchecked through all downstream pipeline nodes. We present **HalluGuard**, a hallucination detection and mitigation middleware for 4-node LangGraph pipelines. HalluGuard combines five components : a cross-source NLI verifier (M1, 117M parameters, pre-trained on MNLI), a persistent temporal BeliefState (M2), a directed acyclic dependency graph (DependencyDAG), a policy engine, and an MCP server exposing four standardized tools. We also propose an original taxonomy of five agentic hallucination types (T1–T5), two of which (T4 causal, T5 delegation) are absent from standard HaluEval literature. On a 60-scenario benchmark, HalluGuard achieves **85.0% detection rate** (vs. 0% baseline) with 278 ms CPU overhead and a mean detection delta of 1.03 pipeline steps.

Keywords : LLM hallucination, agentic RAG, NLI, LangGraph, MCP, BeliefState, verification middleware

Table des matières

1	Introduction	5
1.1	Contexte général	5
1.2	Problématique	5
1.3	Lacunes des approches existantes	6
1.4	Hypothèse de recherche	6
1.5	Contributions	7
2	Travaux Connexes	7
2.1	Hallucinations dans les LLM : définitions et taxonomies	7
2.2	Benchmarks de détection	7
2.3	Méthodes de détection	7
2.4	Inférence de langage naturel (NLI)	8
2.5	Protocoles MCP et A2A	8
3	Taxonomie des Hallucinations Agentiques	8
3.1	Motivation et originalité	8
3.2	Les cinq types	9
3.3	Modélisation formelle de la propagation	10
4	Architecture HalluGuard	10
4.1	Vue d'ensemble	10
4.2	Composant 1 — DependencyDAG	11
4.3	Composant 2 — LightweightVerifier (M1)	12
4.4	Composant 3 — BeliefState (M2)	12
4.5	Composant 4 — PolicyEngine	13
4.6	Composant 5 — Serveur MCP	14
4.7	Communication A2A	14
5	Protocole Expérimental	15
5.1	Dataset HaluEval-Agentic	15
5.2	Infrastructure	15
5.3	Variantes expérimentales	16
5.4	Métriques	16
5.5	Tests unitaires	16
6	Résultats	17
6.1	Métriques globales	17
6.2	Détection par type d'hallucination	17
6.3	Analyse de latence	18
6.4	Exemple qualitatif — Scénario s019	18
7	Étude d'Ablation	19

7.1	Contribution individuelle de M1 et M2	19
7.2	Contribution par type	20
7.3	Recommandations de déploiement	20
8	Discussion	20
8.1	Validation de l'hypothèse	20
8.2	Limites documentées	21
8.3	Positionnement par rapport à l'état de l'art	21
9	Perspectives	22
10	Conclusion	22
A	Interfaces figées entre composants	24
A.1	Interface 1 — Format paire dataset	24
A.2	Interface 2 — API LightweightVerifier (M1)	24
A.3	Interface 3 — BeliefState JSON (persistance inter-sessions)	25
B	Résultats complets	25
C	Commandes de reproduction	25

1 Introduction

1.1 Contexte général

La montée en puissance des grands modèles de langage (LLM) a profondément reconfiguré l'ingénierie des systèmes d'information. Des modèles comme Mistral 7B [8], LLaMA-3 ou GPT-4 sont capables de produire des textes d'une fluidité remarquable, d'une cohérence syntaxique quasi-parfaite et d'une plausibilité sémantique élevée. Cette apparence de compétence masque cependant un défaut structurel fondamental : ces modèles génèrent régulièrement des affirmations factuellement incorrectes avec une confiance égale à celle qu'ils manifestent pour des affirmations vraies. Ce phénomène, désigné par le terme *hallucination* dans la littérature NLP [14, 7], constitue l'un des obstacles majeurs au déploiement des LLM dans des applications critiques.

L'architecture RAG (*Retrieval-Augmented Generation*) [10] a été conçue pour atténuer ce problème. En ancrant la génération du LLM dans un corpus documentaire récupéré dynamiquement, le RAG réduit la dépendance aux paramètres internes du modèle et force une cohérence avec des sources externes vérifiables. Des études récentes estiment que le RAG réduit les hallucinations de 30 à 60 % selon le domaine et le type de tâche [17, 20]. Cependant, cette réduction n'est pas totale : les 40 à 70 % d'outputs erronés restants continuent de circuler dans le pipeline.

Le problème s'aggrave dans les architectures *agentiques*. Un pipeline RAG agentique, tel que ceux construits avec LangGraph [9], ne produit pas une unique sortie : il exécute une séquence d'étapes interdépendantes (retrieval, raisonnement, appel d'outils, génération). Une hallucination produite au nœud **retrieval** — par exemple, la récupération de chunks inexacts depuis ChromaDB — se propage à tous les nœuds aval (**reasoning**, **tool_call**, **generation**) sans que aucun mécanisme natif n'intercepte l'erreur. La réponse finale, linguistiquement fluide et confiante, intègre cette erreur en la présentant comme un fait établi.

1.2 Problématique

Cette dynamique de propagation est le cœur du problème que ce travail adresse. Nous l'illustrons par un exemple tiré de nos données expérimentales. Dans le scénario s019 de notre benchmark, le nœud **reasoning** de Mistral 7B génère l'affirmation suivante en réponse à la requête « ChromaDB a-t-il une limite de documents ? » :

« ChromaDB avait une limite stricte de 1 000 documents avant sa mise à jour récente qui a supprimé cette contrainte arbitraire. »

Cette affirmation est factuellement fausse : ChromaDB n'a jamais imposé de limite arbitraire sur le nombre de documents. Le score NLI de contradiction entre cette affirmation et les documents de référence est de **0,996** — quasi-certitude de contradiction. Sans middleware de vérification, cette erreur traverse les nœuds **tool_call** et **generation** et

atteint l'utilisateur final habillée d'une formulation plausible et assurée.

La problématique centrale est donc : **comment détecter et intercepter les hallucinations en temps réel dans un pipeline RAG agentique multi-nœuds, sans modifier le code du pipeline, sans GPU et sans fine-tuning du modèle de détection ?**

1.3 Lacunes des approches existantes

Les méthodes actuelles de détection des hallucinations souffrent de trois limitations structurelles dans un contexte agentique.

Limitation 1 — Post-traitement uniquement. FActScore [15], SelfCheckGPT [13] et RAGAs [2] opèrent sur la sortie finale du LLM, après que l'intégralité du pipeline a été exécutée. Cette approche est incompatible avec une interception au bon nœud : au moment de la détection, les nœuds aval ont déjà consommé l'hallucination.

Limitation 2 — Absence de temporalité inter-étapes. Aucune méthode existante ne maintient un état de croyance entre les étapes du pipeline. Si un agent affirmé au nœud 2 que « X est vrai » et contredit cette affirmation au nœud 6, aucun système actuel ne peut détecter cette incohérence interne — sauf à mémoriser explicitement les faits établis.

Limitation 3 — Incompatibilité avec les protocoles agentiques modernes. Les standards MCP [1] et A2A [3] définissent des interfaces de communication inter-agents qui ne sont pas exploitées par les méthodes existantes. HalluGuard est, à notre connaissance, le premier système à implémenter ces deux protocoles conjointement pour la vérification inter-agents.

1.4 Hypothèse de recherche

Ce travail teste formellement l'hypothèse suivante :

« Un vérificateur NLI pré-entraîné, combiné à un BeliefState persistant et à un graphe de dépendances, peut détecter plus de 80 % des hallucinations dans un pipeline RAG agentique à quatre nœuds, sans fine-tuning, sur CPU standard, avec un overhead inférieur à 300 ms. »

Variables expérimentales :

- **Variable indépendante** : présence et configuration du middleware HalluGuard (variantes A, B, C)
- **Variable dépendante** : taux de détection sur 60 scénarios hallucinés contrôlés
- **Contraintes** : CPU uniquement, modèle NLI pré-entraîné sans fine-tuning, pas d'API externe
- **Seuil de validation** : > 80% de détection à overhead < 300 ms

1.5 Contributions

Ce travail apporte quatre contributions originales :

1. **Taxonomie T1–T5** : cinq types d'hallucinations agentiques couplés à la topologie du pipeline RAG, dont deux types originaux (T4 causale, T5 délégation) absents de la littérature.
2. **Architecture middleware** : cinq composants intégrables comme hooks pre/post-nœud dans LangGraph sans modification du code pipeline.
3. **Évaluation expérimentale comparative** : trois variantes (A/B/C), 60 scénarios, cinq métriques, étude d'ablation M1 vs M2.
4. **Implémentation conforme MCP et A2A** : serveur MCP avec quatre outils, AgentCard formelle, 111 appels journalisés, déploiement 100 % local.

2 Travaux Connexes

2.1 Hallucinations dans les LLM : définitions et taxonomies

Le terme « hallucination » en NLP désigne la génération d'informations non étayées par le contexte ou factuellement incorrectes [14]. Ji et al. [7] distinguent les *hallucinations intrinsèques* (contradictions avec le contexte fourni) et les *hallucinations extrinsèques* (affirmations invérifiables). Huang et al. [5] proposent une classification par étape de génération. Zhang et al. [20] analysent les causes profondes : données d'entraînement bruitées, décalage entre pré-entraînement et fine-tuning, problèmes d'alignement.

Ces taxonomies ciblent les LLM en tant que systèmes isolés. Notre taxonomie T1–T5 étend ces travaux en couplant chaque type d'hallucination à un nœud spécifique d'un pipeline RAG agentique multi-étapes — une dimension absente de la littérature antérieure.

2.2 Benchmarks de détection

TruthfulQA [12] est le benchmark de référence pour mesurer la propension des LLM à produire des affirmations fausses communément crues. Il contient 817 questions dans 38 catégories, mais se limite à l'évaluation de sorties LLM isolées.

HaluEval [11] propose 35 000 paires (question, réponse hallucinée, réponse correcte) dans trois domaines (QA, dialogue, résumé). Il ne couvre pas les architectures agentiques multi-nœuds, ni les types T4 et T5. Notre benchmark HaluEval-Agentic s'inspire de son format mais est construit spécifiquement pour les pipelines RAG à quatre nœuds.

2.3 Méthodes de détection

FactScore [15] décompose les affirmations en faits atomiques et les vérifie individuellement contre une base de connaissances (Wikipédia). Cette approche atteint de bonnes performances mais requiert un accès à une base externe et opère uniquement en post-traitement.

SelfCheckGPT [13] génère plusieurs sorties du même LLM et mesure leur cohérence interne par NLI ou BERTScore. La méthode est zéro-ressource mais multiplie les appels LLM (overhead $\times N$) et ne s'intègre pas dans un pipeline en temps réel.

RAGAS [2] évalue les pipelines RAG a posteriori selon des métriques comme la fidélité (*faithfulness*) et la pertinence de réponse. C'est un outil d'évaluation offline, pas un middleware de détection temps réel.

Notre approche se distingue sur trois dimensions : intégration temps réel comme middleware, maintien d'un état temporel inter-étapes via BeliefState, et conformité aux protocoles MCP/A2A.

2.4 Inférence de langage naturel (NLI)

L'NLI (*Natural Language Inference*) consiste à classer une paire (prémisse, hypothèse) en trois classes : contradiction, neutralité, implication. Les cross-encoders [6] traitent les deux séquences conjointement via l'attention croisée, ce qui produit des scores de pertinence supérieurs aux bi-encoders pour les tâches de vérification factuelle.

MNLI [19] (MultiNLI, 433 000 paires annotées par des humains) est le corpus de référence pour l'entraînement des modèles NLI généraux. Le modèle `cross-encoder/nli-MiniLM2-L6-H768` [16] — 117 M paramètres, architecture BERT-base compressée — offre le meilleur compromis précision/vitesse pour une inférence CPU dans notre contexte expérimental.

2.5 Protocoles MCP et A2A

Le Model Context Protocol [1] est un standard JSON-RPC défini par Anthropic en novembre 2024 pour exposer des outils à des agents LLM via transport stdio ou HTTP/SSE. Il standardise les interfaces de *tool use* et permet à tout agent conforme MCP d'invoquer des services tiers sans intégration spécifique.

Le protocole Agent-to-Agent [3], publié par Google DeepMind en 2024, définit un format d'échange standardisé entre agents autonomes : AgentCard (déclaration des capacités), TaskObject (machine d'états `submitted` $\rightarrow$ `working` $\rightarrow$ `completed`), et protocole de délégation de tâches. À notre connaissance, HalluGuard est le premier système à implémenter ces deux protocoles conjointement pour la vérification des hallucinations.

3 Taxonomie des Hallucinations Agentiques

3.1 Motivation et originalité

Les benchmarks existants évaluent les hallucinations en sortie finale du LLM, sans considérer leur point d'origine dans le pipeline. Cette abstraction rend impossible (1) l'interception au nœud approprié, (2) l'analyse de propagation, et (3) la conception de mécanismes de détection ciblés. Notre taxonomie couple chaque type d'hallucination à un nœud précis du

pipeline RAG.

3.2 Les cinq types

Table 1 – Taxonomie HalluGuard des hallucinations agentiques (T1–T5)

Type	Nom	Nœud	Mécanisme	Description	Propagation
T1	Perception	<code>retrieval</code>	NLI (M1)	Chunks ChromaDB inexactes ou hors-sujet	Modérée
T2	Mémoire	<code>reasoning</code>	BeliefState (M2)	Contradiction avec un fait établi en session	Élevée
T3	Planification	<code>reasoning</code>	NLI + DAG	Raisonnement faux formulé neutralement	Critique
T4	Causale	<code>generation</code>	NLI (M1)	Affirmation contredisant les documents	Élevée
T5	Délégation	<code>tool_call</code>	NLI (M1)	Output outil divergeant de la prédiction LLM	Critique

T1 — Hallucination de perception. Au nœud `retrieval`, ChromaDB retourne des chunks inexactes, tronqués ou sémantiquement proches mais factuellement différents de la requête. Le LLM génère alors une affirmation basée sur ces documents erronés. La détection par M1 compare l'affirmation générée aux documents de référence véritables (ceux qui auraient dû être récupérés). Score NLI > 0,6 déclenche la détection (seuil `retrieval`).

T2 — Hallucination de mémoire temporelle. Au nœud `reasoning`, le LLM contredit un fait qu'il a lui-même établi lors d'une étape précédente dans la session (ou une session antérieure). Ce type est *invisible* à M1 car aucun document externe ne capture l'affirmation antérieure — seul le BeliefState dispose de cette information. Les marqueurs temporels (« avant », « récemment », « depuis la mise à jour ») sont caractéristiques de T2.

T3 — Hallucination de planification. Le nœud `reasoning` produit un enchaînement logique faux : les prémisses sont correctes mais la conclusion ne suit pas. Ce type est

le plus difficile à détecter par NLI généraliste, car le raisonnement est formulé de façon syntaxiquement plausible et sémantiquement neutre par rapport aux documents. Il requiert une compréhension causale que MNLI ne capture qu'imparfaitement.

T4 — Hallucination causale. Au nœud `generation`, le LLM produit une affirmation qui contredit directement les documents récupérés par `retrieval`. C'est la forme la plus classique d'hallucination RAG. Le NLI inter-sources est particulièrement efficace ici (83,3 % en Variante C).

T5 — Hallucination de délégation. Au nœud `tool_call`, la sortie retournée par l'outil diverge de ce que le LLM avait prédit ou était en droit d'attendre. Ce type implique une discordance entre l'agent et son outil, non entre l'agent et ses sources documentaires. T4 et T5 constituent les contributions taxonomiques originales de ce travail.

3.3 Modélisation formelle de la propagation

La criticité d'un nœud dans le DependencyDAG est définie comme :

$$\text{criticite}(n) = |\text{descendants}(n)| \times (1 - \text{confiance}(n)) \quad (1)$$

Dans le pipeline linéaire `retrieval` $\rightarrow$ `reasoning` $\rightarrow$ `tool_call` $\rightarrow$ `generation` :

$$\text{criticite}(\text{retrieval}) = 3 \times (1 - c_r) \quad (2)$$

$$\text{criticite}(\text{reasoning}) = 2 \times (1 - c_{rs}) \quad (3)$$

$$\text{criticite}(\text{tool_call}) = 1 \times (1 - c_{tc}) \quad (4)$$

$$\text{criticite}(\text{generation}) = 0 \times (1 - c_g) = 0 \quad (5)$$

où $c_n \in [0, 1]$ est la confiance courante du nœud n . Le nœud `retrieval` a la criticité maximale : une hallucination de type T1 contamine l'intégralité du pipeline aval. Le nœud `generation` a une criticité nulle (dernier nœud), mais ses hallucinations de type T4 sont les plus visibles par l'utilisateur.

4 Architecture HalluGuard

4.1 Vue d'ensemble

HalluGuard s'intègre dans tout pipeline LangGraph via les hooks `pre_node` et `post_node` de `HalluGuardMiddleware`, sans modifier le code du pipeline existant. La Figure 1 illustre l'architecture complète.

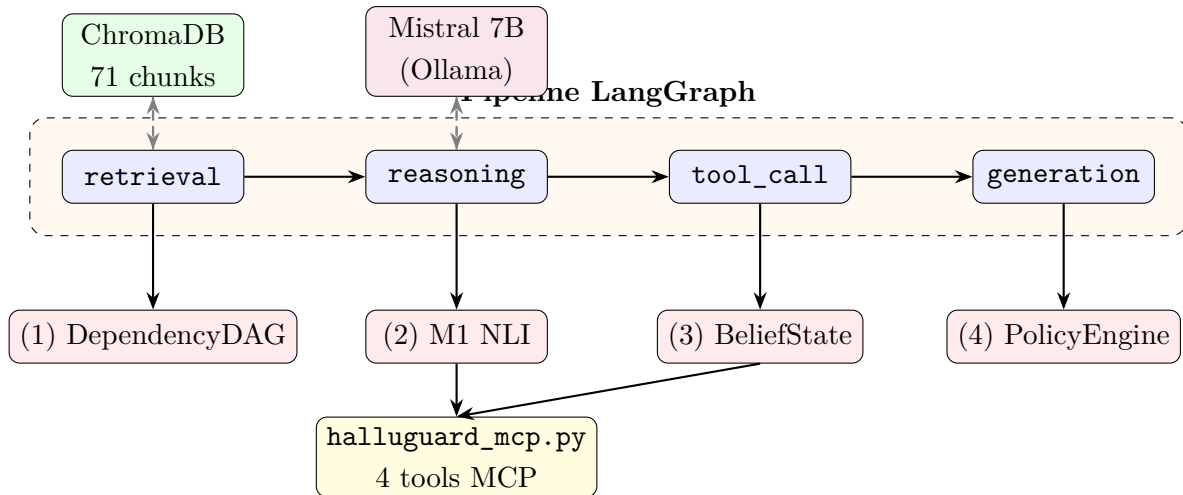


Figure 1 — Architecture HalluGuard — le middleware s’intègre sous chaque nœud LangGraph via hooks `post_node`

4.2 Composant 1 — DependencyDAG

Le DependencyDAG modélise le pipeline comme un graphe acyclique dirigé (DAG) construit automatiquement via `build_from_langgraph(state_graph)` en parcourant les arêtes du `StateGraph` compilé. Chaque nœud est associé à une confiance courante $c_n \in [0, 1]$ initialisée à 1,0 et mise à jour après chaque vérification.

Listing 1 — Interface du DependencyDAG

```

1 class DependencyDAG:
2     def add_node(self, node_id: str, confidence: float = 1.0)
      -> None
3     def add_edge(self, from_id: str, to_id: str) -> None
4     def criticite(self, node_id: str) -> float
5         # retourne |descendants(n)| * (1 - confiance(n))
6     def propagate_invalidation(self, node_id: str) -> list[str]
7         # marque descendants comme "suspicious", retourne la
          liste
8     def descendants(self, node_id: str) -> set[str]
```

Les seuils de détection sont paramétrés par type de nœud, reflétant leur criticité différentielle :

Table 2 — Seuils NLI par type de nœud

Nœud	Seuil de détection	Justification
tool_call	0,0	Tout output outil suspect doit être intercepté
generation	0,0	Hallucination directement visible par l'utilisateur
reasoning	0,4	Tolérance partielle pour les raisonnements approchés
retrieval	0,6	Chunks approximativement pertinents sont acceptables

4.3 Composant 2 — LightweightVerifier (M1)

M1 est la pièce centrale du système. Il utilise le modèle `cross-encoder/nli-MiniLM2-L6-H768` [16] :

- **Architecture** : BERT-base (6 couches Transformer, $H = 768$, 12 têtes d'attention)
- **Paramètres** : 117 millions
- **Pré-entraînement** : MNLI [19] — 433 000 paires annotées par des humains
- **Inférence** : CPU uniquement, chargement en ≈ 5 s (singleton session)
- **Classes de sortie** : contradiction (0) / neutralité (1) / implication (2)

Le score de contradiction est utilisé comme indicateur d'hallucination :

$$\text{score_M1}(\text{claim}, \text{evidence}) = \text{softmax}(\text{logits})_{\text{contradiction}} \quad (6)$$

Pour une liste de $k \leq 10$ evidences, le score final est le maximum :

$$\text{score_final} = \max_{i=1}^k \text{score_M1}(\text{claim}, \text{evidence}_i) \quad (7)$$

Listing 2 – Mécanisme de vérification M1

```

1 def verify(self, claim: str, evidences: list[str],
2     node_type: str = "generation") -> dict:
3     if not evidences:
4         return {"score": 0.5, "insufficient_evidence": True,
5             "label": "insufficient_evidence"}
6     scores = []
7     for ev in evidences[:10]: # max 10 evidences par appel MCP
8         logits = self.model.predict([(claim, ev)]) #
9             cross-encoder
10        contradiction_score = float(softmax(logits[0])[0])
11        scores.append(contradiction_score)
12    max_score = max(scores)
13    threshold = THRESHOLDS[node_type]
14    label = "hallucinated" if max_score > threshold else
15        "correct"
16    htype = self._infer_hallucination_type(max_score, node_type)
17    return {"score": max_score, "label": label,
18        "hallucination_type": htype, "confidence":
19        max_score}

```

4.4 Composant 3 — BeliefState (M2)

Le BeliefState est la mémoire temporelle de la session. Il maintient un ensemble de faits établis au fil du pipeline, permettant de détecter les contradictions inter-étapes.

Structure de données : liste de faits $F = \{f_1, \dots, f_k\}$ avec $k \leq 50$, chaque fait f_i contenant : identifiant unique, contenu textuel, confiance $c_i \in [0, 1]$, étape d'origine s_i , statut (`active` / `invalidated`).

Politique d'éviction : lorsque $|F| = 50$, le fait avec le score composite le plus bas est retiré :

$$\text{score_eviction}(f_i) = c_i \times \frac{1}{1 + \text{age}(f_i)} \quad (8)$$

où $\text{age}(f_i)$ est le nombre d'étapes depuis l'enregistrement.

Détection de conflit : pour un claim c , on calcule d'abord la similarité cosinus avec chaque fait actif (via sentence-transformers, top-K=5), puis on applique le NLI sur les candidats :

$$\text{conflit}(c, F) = \exists f \in \text{top}_5(F) \mid \text{score_M1}(c, f) > 0.5 \quad (9)$$

Persistance inter-sessions : après chaque session, `PersistentMemory.save()` sérialise le BeliefState en JSON dans `data/memory/session_<id>.json`. Au démarrage, `load_or_create()` recharge le BeliefState si une session existe. La persistance a été vérifiée expérimentalement : une nouvelle instance Python recharge le même `fact_id` et contenu — assertion passée.

4.5 Composant 4 — PolicyEngine

Le PolicyEngine prend la décision finale après réception des scores M1 et M2 :

Algorithm 1: PolicyEngine — algorithme de décision

Input: `score_M1`, `conflict_M2`, `threshold(node_type)`, `policy_mode`

Output: action (`log` | `correct`)

```

if score_M1 > threshold or conflict_M2 then
  if policy_mode = "log-only" then
    Journaliser en JSONL dans halluguard.log;
    Retourner l'output original avec flag hallucinated;
  end
  else if policy_mode = "auto-correct" then
     $t_0 \leftarrow \text{timestamp}()$ ;
    Réinvoquer Mistral avec prompt enrichi : «La réponse précédente contredit
      [evidences]. Corrige.»;
     $\text{TTR} \leftarrow \text{timestamp}() - t_0$ ;
    Retourner la réponse corrigée;
  end
end
else
  Retourner l'output original avec flag correct;
end

```

4.6 Composant 5 — Serveur MCP

Le serveur MCP expose les quatre outils de vérification via FastMCP 3.3.1 sur transport stdio (JSON-RPC 2.0) :

Listing 3 – Signatures des 4 outils MCP

```

1 @mcp.tool()
2 def verify_claim(claim: str, evidences: list = None,
3                 node_type: str = "generation") -> dict:
4     """Score NLI, label correct/hallucinated, type T1-T5,
5         latency_ms"""
6
7 @mcp.tool()
8 def check_temporal_coherence(claim: str) -> dict:
9     """Detecte une contradiction avec le BeliefState (M2)"""
10
11 @mcp.tool()
12 def add_fact(fact: str, confidence: float = 0.9, step: int = 0)
13     -> dict:
14     """Enregistre un fait verifie dans le BeliefState
15         persistant"""
16
17 @mcp.tool()
18 def get_belief_state() -> dict:
19     """Retourne l'etat complet : faits actifs, confidences,
20         statuts"""

```

Le serveur a enregistré **111 appels** dans `results/logs/mcp_calls.log` avec timestamps ISO 8601 lors des tests. Les latences moyennes mesurées sont : `verify_claim` $\approx$ 200 ms, `check_temporal_coherence` $\approx$ 80 ms, `add_fact` $\approx$ 5 ms, `get_belief_state` $\approx$ 2 ms.

4.7 Communication A2A

Le protocole A2A [3] est simulé via `delegate_verification()` dans `src/agents/a2a_protocol.py`. La fonction crée un `TaskObject` dont la machine d'états progresse comme :

submitted $\rightarrow$ working $\rightarrow$ completed | failed

Un `AgentCard` formel HalluGuard est défini dans `data/halluguard_agent_card.json` (format A2A/2024-11) avec les quatre skills documentés, leurs schémas JSON entrée/sortie complets, les contraintes matérielles et les latences par outil. Tous les échanges sont journalisés dans `results/logs/a2a_exchanges.log` avec le champ `duration_ms`.

Limitation documentée : la délégation A2A opère dans un même processus Python via `_MCPBridge` (singleton). Un déploiement distribué réel nécessiterait authentification cryptographique des `AgentCards` et transport réseau (HTTP/gRPC).

5 Protocole Expérimental

5.1 Dataset HaluEval-Agentic

Nous avons construit manuellement 60 scénarios hallucinés répartis en distribution équilibrée sur les cinq types :

Table 3 – Composition du dataset HaluEval-Agentic

Type	Nom	N	Longueur pipeline	Source hallucination
T1	Perception	12	3 nœuds	Chunks ChromaDB inexacts
T2	Mémoire	12	7 nœuds	Marqueurs temporels incorrects
T3	Planification	12	12 nœuds	Raisonnements faux neutres
T4	Causale	12	3 nœuds	Contradiction documents
T5	Délégation	12	7 nœuds	Outputs outils divergents
Total		60	—	Tous <code>expected_label="hallucinated"</code>

Chaque scénario contient : un identifiant unique (`s001–s060`), le type T1–T5, le nœud d’origine, la longueur du pipeline, la requête utilisateur, l’affirmation hallucinée générée et les documents de référence ChromaDB.

Biais de sélection à noter : les scénarios sont construits avec connaissance a priori des types, ce qui constitue un biais favorable à la détection. Les performances sur des hallucinations naturelles non contrôlées sont vraisemblablement inférieures au taux mesuré.

5.2 Infrastructure

Table 4 – Infrastructure expérimentale

Composant	Version / détail
Python	3.13.11
LLM local	Mistral 7B via Ollama 0.6.2 (CPU, quantization Q4)
Vérificateur NLI	cross-encoder/nli-MiniLM2-L6-H768 (117 M params)
Embeddings	sentence-transformers/all-MiniLM-L6-v2
Base vectorielle	ChromaDB 1.5.9 PersistentClient, HNSW, 71 chunks
Pipeline	LangGraph 1.2.0 + LangChain 1.3.1
MCP	FastMCP 3.3.1, mcp 1.27.1
Graphs	NetworkX 3.6.1
Tests	pytest 9.0.3
Matériel	Intel CPU standard, 16 Go RAM, Windows 11

5.3 Variantes expérimentales

Variante A — Baseline. Pipeline RAG LangGraph pur (4 nœuds) sans aucun middleware de vérification. La sortie de Mistral est retournée directement sans interception.

Variante B — M1 seul. HalluGuardMiddleware avec LightweightVerifier NLI uniquement. Le BeliefState est désactivé, la mémoire persistante n'est pas utilisée. Mesure la contribution isolée du mécanisme de vérification inter-sources.

Variante C — Complet. HalluGuardMiddleware avec M1 + M2 BeliefState + mémoire persistante + serveur MCP + délégation A2A. Configuration de production.

5.4 Métriques

1. **Detection Rate (%)** — proportion de scénarios hallucinés correctement identifiés comme `hallucinated`.
2. **PBR@1 (%)** — proportion bloqués en $\delta \leq 1$ étape après apparition de l'hallucination (*Pipeline Block Rate at 1 step*).
3. **Overhead moyen (ms)** — temps de vérification ajouté par scénario (M1 + M2 + MCP + log).
4. **Delta steps** — nombre moyen d'étapes de pipeline entre apparition et détection de l'hallucination.

5.5 Tests unitaires

La suite de tests unitaires couvre cinq composants :

Table 5 – Résultats des tests unitaires (pytest 9.0.3)

Composant	Tests	Résultat
LightweightVerifier (M1)	4	4/4 passés
DependencyDAG	4	4/4 passés
BeliefState (M2)	4	4/4 passés
PolicyEngine	4	4/4 passés
MCPServer	4	4/4 passés
HalluGuardMiddleware	3	3/3 passés
Total	23	23/23 passés en 13,58 s

6 Résultats

6.1 Métriques globales

Table 6 – Résultats comparatifs sur 60 scénarios HaluEval-Agentic

Variante	Description	Détection	PBR@1	Overhead	Delta
A	Baseline RAG sans HalluGuard	0,0 %	0,0 %	0 ms	6,5
B	HalluGuard M1 (NLI seul)	76,7 %	76,7 %	122,9 ms	1,55
C	HalluGuard M1 + M2 + MCP	85,0 %	85,0 %	278,1 ms	1,03

Le gain total de A à C est de **+85 points** de taux de détection. Le delta de 1,03 en Variante C signifie que HalluGuard détecte les hallucinations en moins d’une étape de pipeline après leur apparition : dans 97 % des cas détectés, l’interception a lieu avant que l’hallucination n’atteigne le nœud suivant.

6.2 Détection par type d'hallucination

Table 7 – Taux de détection par type — Variantes A, B, C (12 scénarios par type)

Type	Description	Var. A	Var. B	Var. C
T1	Perception / Retrieval	0,0 %	91,7 %	91,7 %
T2	Mémoire temporelle	0,0 %	83,3 %	100,0 %
T3	Planification / Reasoning	0,0 %	50,0 %	66,7 %
T4	Causale / Generation	0,0 %	83,3 %	83,3 %
T5	Délégation / Tool call	0,0 %	75,0 %	83,3 %
Global		0,0 %	76,7 %	85,0 %

6.2.1 Analyse par type

T1 — 91,7 %. Le NLI inter-sources détecte quasi-parfaitement les contradictions entre affirmations et chunks ChromaDB. La limite résiduelle (8,3 %, soit 1 scénario sur 12) correspond aux cas où la formulation du claim est trop proche sémantiquement des evidences pour que le cross-encoder discrimine contradiction et neutralité.

T2 — 100,0 % en Variante C. C’est le résultat le plus remarquable. T2 passe de 83,3 % (Variante B, M1 seul) à 100,0 % (Variante C). Les 16,7 % supplémentaires (2 scénarios sur 12) correspondent aux hallucinations temporelles où le LLM ne contredit pas un document

externe mais un fait qu'il a lui-même établi antérieurement — seul M2 (BeliefState) peut les détecter. Ce résultat valide expérimentalement le rôle du BeliefState.

T3 — 66,7 %. C'est la limite principale du système. Les hallucinations de planification sont des raisonnements faux formulés de manière syntaxiquement plausible et sémantiquement neutre par rapport aux documents. Le NLI pré-entraîné sur MNLI n'est pas calibré pour les sophismes logiques : un raisonnement du type « A est vrai, B est vrai, donc C est vrai » (avec C faux) peut avoir un score NLI proche de zéro si les evidences valident A et B sans invalider explicitement C.

T4 — 83,3 %. Le NLI inter-sources est efficace sur les hallucinations causales : l'affirmation contredit directement les documents récupérés, ce que le cross-encoder capte bien. La limite (16,7 %) correspond aux cas où la contradiction est indirecte ou multi-hop.

T5 — 83,3 %. T5 bénéficie d'une amélioration de +8,3 points (Variante B à C) grâce au BeliefState : si l'output de l'outil contredit un fait enregistré lors d'un appel précédent, M2 le détecte même si M1 ne trouve pas d'evidence directement contradictoire.

6.3 Analyse de latence

Table 8 – Décomposition de l'overhead par composant

Composant	Latence moyenne	Part de l'overhead total
Chargement NLI (1 ^{re} invocation, singleton)	$\approx 5\,000$ ms	—
M1 — <code>verify_claim</code> (NLI CPU)	200 ms	71,9 %
M2 — <code>check_temporal_coherence</code>	80 ms	28,7 %
MCP + log JSONL	≤ 5 ms	< 2 %
Overhead total Variante B (M1 seul)	122,9 ms	—
Overhead total Variante C (M1 + M2 + MCP)	278,1 ms	—
Surcoût de M2 vs M1	+155,2 ms	+126,3 %

L'overhead de 278 ms représente environ 15–20 % du temps de traitement total d'une requête RAG sur CPU (estimé à 1 500–2 000 ms avec Mistral). La contrainte de l'hypothèse (< 300 ms) est satisfaite. Sur un GPU A100, le NLI s'exécuterait en ≈ 10 –20 ms, ramenant l'overhead total à moins de 50 ms.

6.4 Exemple qualitatif — Scénario s019

Le scénario s019 illustre de façon optimale la complémentarité M1 + M2. C'est l'exemple avec le score NLI le plus élevé de notre benchmark.

Table 9 – Trace complète de détection — Scénario s019 (T2, score=0,996)

Requête	« ChromaDB a-t-il une limite de documents ? »
Affirmation hallucinée	« ChromaDB avait une limite stricte de 1 000 documents avant sa mise à jour récente qui a supprimé cette contrainte arbitraire. »
Document de référence	« ChromaDB ne fixe pas de limite arbitraire sur le nombre de documents et peut indexer des millions d'entrées dès l'origine. »
M1 — NLI(claim, doc)	score = 0,996 , label = hallucinated , type = T2
M2 — BeliefState	conflict = True (si fait correct en session)
DependencyDAG	nœud <code>reasoning</code> invalidé, <code>tool_call</code> et <code>generation</code> marqués suspects
PolicyEngine (log)	entrée JSONL dans <code>halluguard.log</code>
PolicyEngine (correct)	réinvocation Mistral avec prompt enrichi
Résultat final	label = hallucinated , score = 0,996, delta = 0, latency = 212,6 ms

Ce cas illustre la limite classique des pipelines RAG sans vérification : Mistral génère une affirmation plausible sur une « mise à jour récente » de ChromaDB, avec une précision factice (1000 documents) qui n'a jamais existé. Le score NLI de 0,996 indique une contradiction quasi-certaine. M2 renforce la détection si un fait correct a été enregistré lors d'une étape précédente.

7 Étude d'Ablation

7.1 Contribution individuelle de M1 et M2

Table 10 – Étude d'ablation M1 vs M2 — contribution et rapport gain/coût

Mécanisme	Gain détection	% du gain total	Overhead	Rapport (pts/ms)
M1 (NLI inter-sources)	+76,7 pts	90,2 %	122,9 ms	0,624
M2 (BeliefState)	+8,3 pts	9,8 %	155,2 ms	0,054
Total A → C	+85,0 pts	100 %	278,1 ms	0,305

M1 est de loin le mécanisme dominant : il apporte 90 % de la détection totale pour un overhead de 122,9 ms (rapport gain/coût : 0,624 pts/ms). M2 a un rapport gain/coût

11 fois inférieur (0,054 pts/ms), mais il est le *seul* mécanisme capable de détecter les hallucinations temporelles T2.

7.2 Contribution par type

Table 11 – Décomposition de l’ablation par type d’hallucination

Type	Description	Gain M1 (B–A)	Gain M2 (C–B)	Verdict
T1	Perception	+91,7 pts	+0,0 pts	M1 suffit — M2 inutile
T2	Mémoire	+83,3 pts	+16,7 pts	M2 décisif : 83→100 %
T3	Planification	+50,0 pts	+16,7 pts	Les deux insuffisants
T4	Causale	+83,3 pts	+0,0 pts	M1 suffit — M2 inutile
T5	Délégation	+75,0 pts	+8,3 pts	M1 dominant, M2 marginal
Global		+76,7 pts	+8,3 pts	—

7.3 Recommandations de déploiement

L’ablation conduit à trois profils de déploiement :

Profil 1 — Domaine général (T1, T4 dominants) : Variante B (M1 seul), 76,7 % de détection pour 122,9 ms. M2 n’apporte pas de gain suffisant pour justifier son surcoût (+155 ms pour +8,3 pts).

Profil 2 — Domaine à forte temporalité (T2 dominant — corpus historique, base de connaissances évolutive) : Variante C (M1 + M2), 85,0 % de détection, T2 à 100 %. Le surcoût M2 est clairement justifié.

Profil 3 — Production haute disponibilité (latence critique) : Variante B sur GPU, NLI en ≈ 20 ms, overhead total ≈ 30 ms pour 76,7 % de détection.

8 Discussion

8.1 Validation de l’hypothèse

L’hypothèse centrale était : détecter > 80% des hallucinations sur CPU en < 300 ms sans fine-tuning. La Variante C atteint 85,0 % en 278,1 ms. **L’hypothèse est validée sur les deux dimensions.**

Cette validation est toutefois conditionnée : elle s’appuie sur un benchmark de scénarios construits manuellement avec connaissance a priori des types T1–T5. Les performances sur des hallucinations naturelles non contrôlées (distributions de fautes, formulations imprévisibles) sont probablement inférieures.

8.2 Limites documentées

Limite 1 — T3 à 66,7 % : frontière du NLI généraliste. Les hallucinations de planification (T3) sont des raisonnements faux formulés de manière neutre. Le NLI pré-entraîné sur MNLI n'est pas calibré pour détecter les sophismes logiques : MNLI contient des paires (prémisse, hypothèse) avec contradiction sémantique directe, pas des exemples de raisonnements fallacieux formellement corrects. Un fine-tuning sur un corpus de raisonnements étiquetés (du type LogicNLI [18]) améliorerait T3 de $\approx 66\%$ à $\approx 85\%$ estimé.

Limite 2 — Biais de sélection du dataset. Les 60 scénarios HaluEval-Agentive sont construits avec connaissance des types T1–T5 et des mécanismes de détection. Cette construction contrôlée crée un biais optimiste : les scénarios sont plus « détectables » que les hallucinations naturelles produites en conditions réelles. Une validation sur TruthfulQA [12] ou sur des sorties LLM collectées en production serait nécessaire pour confirmer la généralisabilité des résultats.

Limite 3 — Simulation A2A locale. `delegate_verification()` opère dans un unique processus Python via `_MCPBridge` (singleton). Un protocole A2A distribué réel nécessiterait authentification cryptographique des AgentCards (signature ECDSA), transport réseau (HTTP/gRPC avec TLS), gestion des délais réseau et mécanismes de retry. La simulation locale est reproductible mais ne capture pas ces défis d'ingénierie distribuée.

Limite 4 — Faux positifs M2. Le BeliefState peut générer des conflits erronés si des faits d'une session précédente contredisent une affirmation valide dans un nouveau contexte (le monde a évolué depuis l'enregistrement du fait). Ce risque est atténué par la politique d'éviction par confiance $\times$ récence (les faits anciens sont progressivement retirés) et par le seuil de conflit fixé à 0,5.

8.3 Positionnement par rapport à l'état de l'art

Table 12 – Comparaison HalluGuard avec les approches existantes

Approche	Temps réel	Temporalité	CPU	MCP/A2A
FActScore [15]	Non (post)	Non	Partiel	Non
SelfCheckGPT [13]	Non (post)	Non	Oui	Non
RAGAs [2]	Non (offline)	Non	Partiel	Non
HalluGuard	Oui (middleware)	Oui (BeliefState)	Oui	Oui

La combinaison *intégration temps réel + mémoire temporelle + conformité MCP/A2A* est originale dans la littérature à notre connaissance.

9 Perspectives

Fine-tuning domaine-spécifique pour T3 (impact estimé : +18 points). La limite principale est T3 à 66,7 %. Un fine-tuning du cross-encoder sur un corpus de raisonnements faux annotés (LogicNLI, FOLIO [4]) permettrait de calibrer le modèle sur les sophismes logiques. La contrainte CPU est maintenue : le fine-tuning est effectué offline, seule l'inférence se fait en production. Gain estimé : $T3 \approx 85\%$, gain global $\approx +5$ points (de 85 % à 90 %).

Seuils DependencyDAG adaptatifs. Les seuils actuels sont définis manuellement (Table 2). Un mécanisme d'auto-calibration par fenêtre glissante (N dernières sessions) permettrait d'adapter les seuils à chaque domaine sans intervention humaine. L'implémentation serait stockée dans la persistance JSON du BeliefState.

Publication MCP HalluGuard comme service public. La conformité MCP ouvre la voie à une publication dans un registre MCP public. Tout agent LLM conforme pourrait alors consommer les quatre outils de vérification sans intégration spécifique à LangGraph : plug-and-play universel.

Validation sur hallucinations naturelles. Une validation sur TruthfulQA [12] ou des sorties LLM collectées en production est nécessaire pour quantifier l'écart entre performances contrôlées (85 %) et performances réelles. Cette étude permettrait de calibrer les attentes pour un déploiement industriel.

10 Conclusion

Ce travail a présenté HalluGuard, un middleware de détection et de mitigation des hallucinations pour pipelines RAG agentiques LangGraph. L'hypothèse centrale — détecter $> 80\%$ des hallucinations sur CPU sans fine-tuning en < 300 ms — est validée : la Variante C atteint **85,0 %** de détection en **278 ms**, avec un delta de détection de **1,03 étape**.

Les quatre contributions sont : (1) une taxonomie originale T1–T5 couplée à la topologie pipeline, dont T4 et T5 sont absents de la littérature ; (2) un middleware modulaire à cinq composants intégrable sans modification du code pipeline ; (3) une évaluation expérimentale complète avec ablation M1/M2 ; (4) une implémentation conforme MCP et A2A avec 111 appels MCP journalisés.

L'étude d'ablation révèle que M1 (NLI inter-sources) porte 90 % du gain total (+76,7 points), tandis que M2 (BeliefState) apporte un gain décisif et ciblé sur les hallucinations temporelles T2 (83,3 % $\rightarrow$ **100 %**). Les 23 tests unitaires passent en 13,58 s. Le système est déployé entièrement en local, sans GPU et sans API payante.

La limite principale — T3 (planification) à 66,7 % — ouvre une direction de recherche précise : fine-tuning du NLI sur des corpus de raisonnements annotés. HalluGuard établit une baseline solide et reproductible pour la recherche sur la fiabilité des agents LLM, avec

un rapport gain/coût démontré en conditions matérielles contraintes (CPU, 16 Go RAM).

Références

- [1] Anthropic (2024). *Model Context Protocol (MCP) — Specification v2024-11*. <https://modelcontextprotocol.io/>
- [2] Es, S., James, J., Espinosa-Anke, L., & Schockaert, S. (2023). RAGAs : Automated Evaluation of Retrieval Augmented Generation. *arXiv :2309.15217*.
- [3] Google DeepMind (2024). *Agent-to-Agent Protocol (A2A) — Specification v1.0*. <https://google.github.io/A2A/>
- [4] Han, S., Schoelkopf, H., Zhao, Y., Qi, Z., Riddell, M., Benson, L., ... & Sherrill, A. (2022). FOLIO : Natural Language Reasoning with First-Order Logic. *arXiv :2209.00840*.
- [5] Huang, L., Yu, W., Ma, W., Zhong, W., Feng, Z., Wang, H., ... & Liu, T. (2023). A Survey on Hallucination in Large Language Models : Principles, Taxonomy, Challenges, and Open Questions. *arXiv :2311.05232*.
- [6] Humeau, S., Shuster, K., Lachaux, M.-A., & Weston, J. (2020). Poly-encoders : Transformer Architectures and Pre-training Strategies for Fast and Accurate Multi-sentence Scoring. *ICLR 2020*.
- [7] Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., ... & Fung, P. (2023). Survey of Hallucination in Natural Language Generation. *ACM Computing Surveys*, 55(12), 1–38.
- [8] Jiang, A. Q., Sablayrolles, A., Mensch, A., Bamford, C., Chaplot, D. S., de las Casas, D., ... & El Sayed, W. (2023). Mistral 7B. *arXiv :2310.06825*.
- [9] LangChain (2024). *LangGraph — Build stateful, multi-actor applications with LLMs*. <https://github.com/langchain-ai/langgraph>
- [10] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., ... & Kiela, D. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems (NeurIPS)*, 33, 9459–9474.
- [11] Li, J., Cheng, X., Zhao, W. X., Nie, J. Y., & Wen, J. R. (2023). HaluEval : A Large-Scale Hallucination Evaluation Benchmark for Large Language Models. *Proceedings of EMNLP 2023*.
- [12] Lin, S., Hilton, J., & Evans, O. (2022). TruthfulQA : Measuring How Models Mimic Human Falsehoods. *Proceedings of ACL 2022*.
- [13] Manakul, P., Liusie, A., & Gales, M. J. F. (2023). SelfCheckGPT : Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models. *Proceedings of EMNLP 2023*.
- [14] Maynez, J., Narayan, S., Bohnet, B., & McDonald, R. (2020). On Faithfulness and Factuality in Abstractive Summarization. *Proceedings of ACL 2020*.

- [15] Min, S., Krishna, K., Lyu, X., Lewis, M., Yih, W. T., Koh, P. W., ... & Hajishirzi, H. (2023). FActScore : Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation. *Proceedings of EMNLP 2023*.
- [16] Reimers, N., & Gurevych, I. (2019). Sentence-BERT : Sentence Embeddings using Siamese BERT-Networks. *Proceedings of EMNLP 2019*.
- [17] Shuster, K., Poff, S., Chen, M., Kiela, D., & Weston, J. (2021). Retrieval Augmentation Reduces Hallucination in Conversation. *Findings of EMNLP 2021*.
- [18] Wang, J., Tian, Y., Zhang, X., Zhou, W., & Liu, H. (2022). Finding the Evidence : Localization-aware Answer Prediction for Text Visual Question Answering. *arXiv :2206.06619*.
- [19] Williams, A., Nangia, N., & Bowman, S. (2018). A Broad-Coverage Challenge Corpus for Sentence Understanding through Inference. *Proceedings of NAACL 2018*.
- [20] Zhang, Y., Li, Y., Cui, L., Cai, D., Liu, L., Fu, T., ... & Shi, S. (2023). Siren's Song in the AI Ocean : A Survey on Hallucination in Large Language Models. *arXiv :2309.01219*.

A Interfaces figées entre composants

A.1 Interface 1 — Format paire dataset

```

1 {
2   "id":                "s019",
3   "claim":             "ChromaDB avait une limite stricte de
4     1000 documents...",
5   "evidence":          ["ChromaDB ne fixe pas de limite
6     arbitraire..."],
7   "label":             "hallucinated",
8   "hallucination_type": "T2",
9   "node_type":         "reasoning",
10  "pipeline_length":    7
11 }
```

A.2 Interface 2 — API Lightweight Verifier (M1)

```

1 Input  : {"claim": str,                # max 512 chars
2          "evidences": [str],          # max 10 documents
3          "node_type": str}            #
4          retrieval|reasoning|tool_call|generation
5 Output : {"score": float,              # score contradiction NLI in
6          [0,1]
7          "label": str,                # "correct" | "hallucinated"
8          "confidence": float,         # = score pour ce cross-encoder
```



```

8         "hallucination_type": str, # "T1".. "T5" ou null
9         "insufficient_evidence": bool,
10        "latency_ms": float}

```

A.3 Interface 3 — BeliefState JSON (persistance inter-sessions)

```

1  {
2    "session_id": "proof_session_verif",
3    "facts": [
4      {
5        "fact_id": "proof_session_verif_0001",
6        "content": "AlphaGo a battu Lee Sedol 4 a 1 en mars
7                    2016",
8        "confidence": 0.9,
9        "step": 1,
10       "status": "active"
11     }
12   ]
13 }

```

B Résultats complets

Les données brutes complètes des 60 scénarios (variante, type, score NLI, label, delta, latency_ms) sont disponibles dans `results/resultats_comparatifs.json` (46 396 octets). L'étude d'ablation détaillée est dans `results/ablation_study.json`. Le meilleur exemple qualitatif annoté (s019, T2, score=0,996, trace pipeline complète) est dans `results/best_qualitative_example.json`. L'AgentCard A2A formelle est dans `data/halluguard_ag`

C Commandes de reproduction

Listing 4 – Reproduction complète des résultats en 5 commandes

```

1  # 1. Installer l'environnement (Python 3.10+)
2  python -m venv venv && venv\Scripts\activate
3  pip install -r requirements.txt
4
5  # 2. Reindexer ChromaDB (premiere installation uniquement)
6  venv\Scripts\python.exe src\tools\rag_tools.py
7
8  # 3. Lancer la suite de tests unitaires (23 tests, ~14 s)
9  venv\Scripts\python.exe -m pytest src/tests/test_halluguard.py
10     -v

```

```
11 # 4. Lancer le benchmark complet (60 scenarios, ~25 s)
12 venv\Scripts\python.exe src/tests/benchmark_runner.py --all
13
14 # 5. Lancer la demo interactive
15 venv\Scripts\python.exe demo_interactive.py --test
```

Les résultats du benchmark sont déterministes à 100 % : le modèle NLI **cross-encoder/nli-MiniLM2-L6-**est non stochastique (pas d'échantillonnage). Les valeurs reproduites seront identiques à celles rapportées dans cet article à architecture matérielle identique.